



Prepared for

Deep Learning

Software and Knowledge Engineering

Department of Computer Engineering

Kasetsart University

Present By

6510545438 Thanutham Chonsongkram

6510545497 Tanapoom Srikaewkheaw

Present To

Prof. Paruj Ratanaworabhan

1. Final project topic

Skin Lesion Classification Using a Custom Convolutional Neural Network

This project focuses on classifying skin lesion images using deep learning. The main goal is to build a custom Convolutional Neural Network (CNN) that can classify dermoscopic skin lesion images into seven different categories. The project uses the HAM10000 dataset, which contains skin lesion images labeled by lesion type.

2. Why is this topic interesting? Why did you choose this topic as your final project?

Because it is a real-world computer vision problem related to healthcare. Skin diseases can appear in many different forms, colors, sizes, and textures. Some types of lesions may look very similar to each other, which makes classification difficult even for humans without medical expertise.

We chose this topic because it is challenging and suitable for deep learning. Unlike simple image classification tasks, skin lesion classification requires the model to learn subtle visual patterns, such as irregular borders, color variations, and texture differences. These details are difficult to describe manually using simple rules.

Another reason for choosing this topic is that it has practical value. A deep learning model for skin lesion classification can be used as a supporting tool for preliminary screening or educational purposes. However, the model in this project is not intended to replace medical professionals. It is developed only as an academic deep learning project.

3. Why does this topic require deep learning to solve the problem? Compare it with other approaches to solving this problem. What are the strengths and weaknesses of the deep learning approach?

Because the input data is image-based and contains complex visual patterns. Skin lesions can differ in color, shape, border, and texture. These features are difficult to define manually. A CNN can automatically learn useful features from images, starting from simple edges and colors in early layers to more complex lesion patterns in deeper layers.

Compared with traditional approaches, deep learning is more suitable for this problem because it does not require manual feature engineering. Traditional machine learning models, such as Random Forest, usually require handcrafted features such as HOG, color histograms, or texture descriptors. These features may not fully capture the complexity of skin lesion images.

Comparison of Approaches

Approach	Description	Strengths	Weaknesses
Rule-based image processing	Uses manually designed rules such as color thresholding, shape detection, or border analysis	Easy to understand and explain	Not flexible; performs poorly when lighting, color, or lesion appearance varies
Traditional machine learning	Uses handcrafted features such as HOG, texture, or color features with classifiers such as Random Forest or SVM	Faster to train and easier to compare as a baseline	Depends heavily on feature extraction quality; may miss complex image patterns
Deep learning using CNN	Learns features directly from images through convolutional layers	Automatically learns complex visual features; suitable for image classification	Requires more data, more computation, and careful training to avoid overfitting

Strengths of the Deep Learning Approach

The main strength of deep learning is its ability to learn image features automatically. CNN layers can detect different visual characteristics of skin lesions, such as edges, color patterns, textures, and irregular shapes. Deep learning also works well with data augmentation, which helps improve generalization by exposing the model to variations in image orientation, brightness, and color.

Weaknesses of the Deep Learning Approach

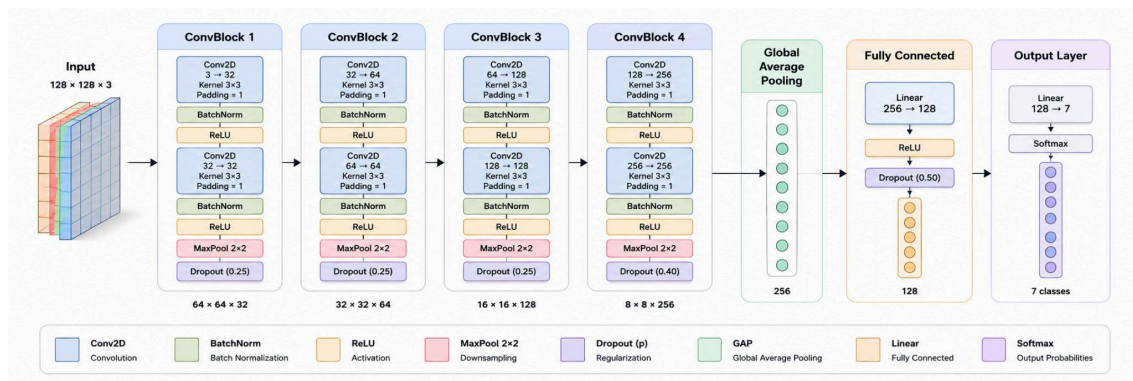
The main weakness is that deep learning requires more computational resources and more training data than traditional methods. It can also overfit if the dataset is limited or imbalanced. In this project, the dataset has class imbalance, so we used techniques such as weighted sampling and weighted loss to reduce the effect of majority classes dominating the training process.

4. Explain the deep learning architecture used, such as feedforward NN, CNN, RNN, GAN, or VAE. Draw a diagram clearly showing the number of nodes, weights, biases, connections, and activation functions.

The deep learning architecture used in this project is a **custom Convolutional Neural Network (CNN)**. CNN is suitable for image classification because it can extract spatial features from images using convolutional filters.

The input image size is $128 \times 128 \times 3$, where 128×128 represents the image height and width, and 3 represents the RGB color channels. The model has four convolutional blocks, followed by global average pooling and fully connected layers.

CNN Architecture Diagram



Explanation of Nodes, Weights, Biases, Connections, and Activation Functions

Each convolutional layer contains learnable filters. These filters act as weights that slide over the image to detect local patterns. Each filter also has a bias term. The convolution operation connects local regions of the previous layer to feature maps in the next layer.

The ReLU activation function is used after convolutional layers and the first fully connected layer. ReLU introduces non-linearity into the network and allows the model to learn complex relationships in image data.

Batch normalization is used to stabilize training and reduce internal distribution shift. MaxPooling reduces the spatial size of feature maps, making the model more efficient and helping it focus on the most important features. Dropout is used to reduce overfitting by randomly disabling some neurons during training.

The final output layer has 7 nodes, where each node represents one skin lesion class.

5. Clearly explain the PyTorch or TensorFlow code, as well as any other code used to connect with other models. Which part handles the data? Which part creates the model? Which part performs training? Etc.

The project code is implemented using PyTorch. The code is divided into several main parts: dataset preparation, data loading, model creation, training, evaluation, and model comparison.

5.1 Dataset Download and Preparation

The dataset is downloaded from Kaggle using the HAM10000 dataset. The metadata file is loaded using pandas. Each image is matched with its file path and lesion label.

The labels are mapped into numerical classes as follows:

```
label_map = {  
    'nv': 0, 'mel': 1, 'bkl': 2, 'bcc': 3,  
    'akiec': 4, 'vasc': 5, 'df': 6  
}
```

The dataset is then split into training, validation, and test sets using stratified splitting. Stratified splitting is used so that each dataset split keeps a similar class distribution.

5.2 Custom Dataset Class

The data handling part is implemented using a custom PyTorch **Dataset** class named **SkinLesionDataset**. This class loads each image, converts it to RGB, applies image transformations, and returns the image tensor with its label.

This part handles the input pipeline for the CNN model.

5.3 Data Transformation and Augmentation

The training images are resized to 128×128 pixels. Data augmentation is applied to the training set to improve generalization. The augmentation includes:

- Random horizontal flip
- Random vertical flip
- Random rotation
- Color jitter
- Normalization

The validation and test sets only use resizing, tensor conversion, and normalization because they should represent real evaluation data without random augmentation.

5.4 DataLoader and Class Imbalance Handling

The project uses PyTorch **DataLoader** to load images in batches. The batch size is 32.

Because the HAM10000 dataset is imbalanced, the project uses **WeightedRandomSampler**. This sampler gives higher sampling probability to minority classes. In addition, the loss function also uses class weights to reduce the bias toward majority classes.

5.5 Model Creation

```
class ConvBlock(nn.Module):
    def __init__(self, in_ch, out_ch, dropout=0.25):
        super().__init__()
        self.block = nn.Sequential(
            nn.Conv2d(in_ch, out_ch, 3, padding=1),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_ch, out_ch, 3, padding=1),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2, 2),
            nn.Dropout2d(dropout)
        )
    def forward(self, x):
        return self.block(x)
```

```
class SkinLesionCNN(nn.Module):
    def __init__(self, num_classes=7):
        super().__init__()
        self.features = nn.Sequential(
            ConvBlock(3, 32, dropout=0.25),
            ConvBlock(32, 64, dropout=0.25),
            ConvBlock(64, 128, dropout=0.25),
            ConvBlock(128, 256, dropout=0.40),
        )
        self.global_avg_pool = nn.AdaptiveAvgPool2d(1)
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(256, 128),
            nn.ReLU(inplace=True),
            nn.Dropout(0.5),
            nn.Linear(128, num_classes)
        )
    def forward(self, x):
        x = self.features(x)
        x = self.global_avg_pool(x)
        return self.classifier(x)
```

ConvBlock defines the repeated convolutional structure used in the CNN. Each block contains convolutional layers, batch normalization, ReLU activation, max pooling, and dropout.

SkinLesionCNN combines four ConvBlocks, global average pooling, and fully connected layers to produce the final prediction.

5.6 Training Code

```
def train_one_epoch(model, loader, criterion, optimizer, device):
    model.train()
    running_loss, correct, total = 0.0, 0, 0
    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item() * images.size(0)
        correct += outputs.max(1)[1].eq(labels).sum().item()
        total += labels.size(0)
    return running_loss / total, correct / total
```

This function performs the following steps:

1. Set the model to training mode.
2. Load a batch of images and labels.
3. Move the data to GPU or CPU.
4. Clear previous gradients using **optimizer.zero_grad()**.
5. Pass images through the model.
6. Compute loss using CrossEntropyLoss.
7. Perform backpropagation using **loss.backward()**.
8. Update model weights using **optimizer.step()**.
9. Calculate training loss and accuracy.

5.7 Validation Code

```
def evaluate(model, loader, criterion, device):
    model.eval()
    running_loss, correct, total = 0.0, 0, 0
    with torch.no_grad():
        for images, labels in loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            loss = criterion(outputs, labels)
            running_loss += loss.item() * images.size(0)
            correct += outputs.max(1)[1].eq(labels).sum().item()
            total += labels.size(0)
    return running_loss / total, correct / total
```

This function sets the model to evaluation mode and disables gradient calculation using **torch.no_grad()**. It calculates validation loss and validation accuracy without updating the model weights.

5.8 Evaluation Code

After training, the model is tested on the test set. The code collects all predictions and true labels, then calculates classification metrics such as accuracy, precision, recall, and F1-score.

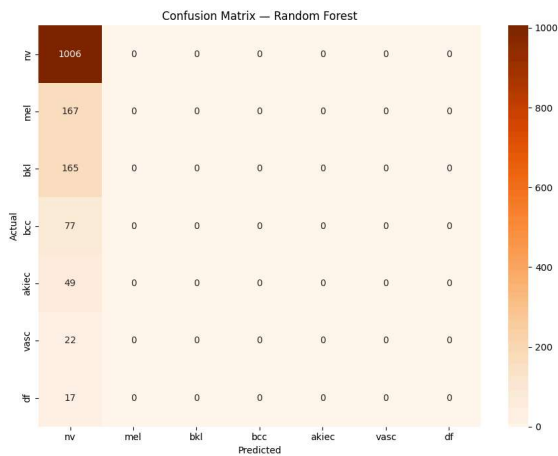
The project also generates a classification report and confusion matrix.

5.9 Baseline Model and Model Comparison

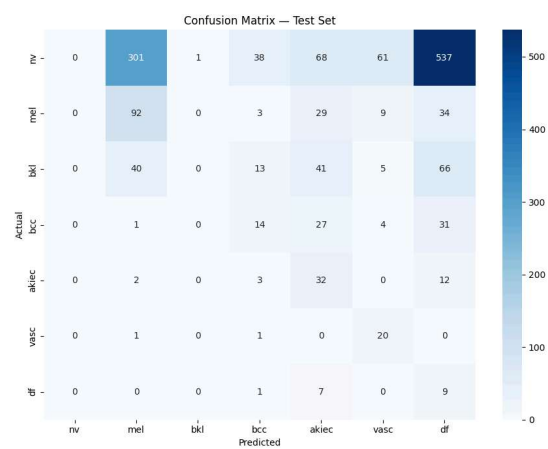
In addition to the custom CNN, the project includes a traditional machine learning baseline using Random Forest with HOG features. The baseline extracts HOG features from images and trains a Random Forest classifier.

The purpose of the baseline is to compare traditional machine learning with deep learning. A separate comparison notebook loads the saved results from both models and creates charts comparing accuracy, precision, recall, F1-score, and per-class F1-score.

Baseline Model (random forest)



Comparison Model



6. Submit a publicly accessible link to the PyTorch or TensorFlow code. This should preferably be a GitHub public repo link.

<https://github.com/Skiffet/skin-lesion-classification.git>

7. Explain the training method for the selected deep learning network, including the relevant dataset and its source.

The selected deep learning network is trained using supervised learning. Each input image has a corresponding label that indicates the skin lesion class. The model learns by comparing its predicted class with the true label and updating its weights to reduce classification error.

Dataset

This project uses the **HAM10000 dataset**, also known as “Human Against Machine with 10000 training images.” The dataset contains **10,015 dermoscopic images** of skin lesions. Each image belongs to one of seven lesion categories.

Dataset source:

<https://www.kaggle.com/datasets/kmader/skin-cancer-mnist-ham10000>

Train, Validation, and Test Split

```
train_df, temp_df = train_test_split(metadata, test_size=0.30, stratify=metadata['label'], random_state=42)
val_df, test_df = train_test_split(temp_df, test_size=0.50, stratify=temp_df['label'], random_state=42)
print(f'Train: {len(train_df)} | Val: {len(val_df)} | Test: {len(test_df)}')
```

Training set: 70%
Validation set: 15%
Test set: 15%

Training Configuration

Setting	Value
Framework	Pytorch
Input image size	128 × 128 RGB
Batch size	32
Number of epochs	30
Optimizer	Adam
Learning rate	0.001
Weight decay	0.0001
Loss function	Weighted CrossEntropyLoss
Scheduler	ReduceLROnPlateau
Class imbalance handling	WeightedRandomSampler and class-weighted loss

8. Explain the evaluation of the model. Show the loss from training and the appropriate evaluation metrics, such as accuracy, precision, or recall.

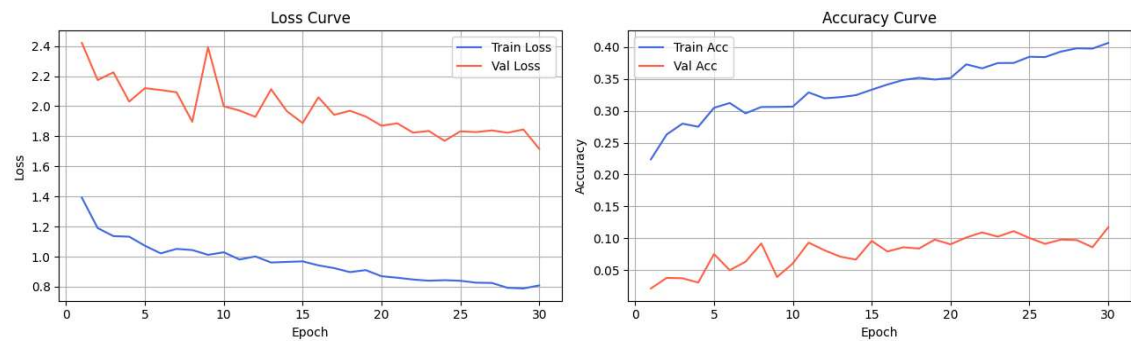
The model is evaluated using the test set after training is complete. The evaluation focuses on both overall performance and per-class performance.

Because the dataset is imbalanced, accuracy alone is not enough. Therefore, the project also uses weighted precision, weighted recall, weighted F1-score, per-class F1-score, and confusion matrix.

Evaluation Metrics

Metric	Meaning
Accuracy	The percentage of correctly classified images.
Precision	How many predicted positive results are correct.
Recall	How many actual class samples are correctly detected.
F1-score	The harmonic mean of precision and recall.
Confusion Matrix	Shows correct and incorrect predictions for each class.

Training and Validation Loss



Test Results

Metric	Custom CNN Result
Test Accuracy	0.1111
Weighted Precision	0.0414
Weighted Recall	0.1111
Weighted F1-score	0.0568

Baseline Comparison

Metric	Random Forest	Custom CNN Result
Test Accuracy	0.6693	0.1111
Weighted Precision	0.4480	0.0414
Weighted Recall	0.6693	0.1111
Weighted F1-score	0.5367	0.0568

Discussion of Results

The CNN is expected to perform better than the Random Forest baseline because CNN can learn features directly from images. Random Forest depends on handcrafted HOG features, which may not fully capture the color, border, and texture patterns of skin lesions.

However, if the CNN does not significantly outperform the baseline, possible reasons include class imbalance, limited image resolution, similarity between lesion classes, or insufficient training time.

9. Explain the reference articles and related work.

This project is related to research in medical image classification and convolutional neural networks. The most important related work is the HAM10000 dataset paper, which introduced a large collection of dermoscopic skin lesion images for machine learning research.

Related Work

The HAM10000 dataset was created to support research on automated skin lesion classification. It contains dermoscopic images from multiple lesion categories and is widely used as a benchmark dataset for skin disease image classification.

Convolutional Neural Networks have been widely used in medical image analysis because they can automatically learn visual features from image data. CNN-based models have shown strong performance in tasks such as skin lesion classification, tumor detection, X-ray classification, and retinal image analysis.

Traditional machine learning approaches, such as Random Forest or Support Vector Machine, have also been used for image classification. However, these methods usually require handcrafted features such as HOG, color histograms, or texture descriptors. In contrast, CNNs learn features automatically from raw images.

References

[1] Tschandl, P., Rosendahl, C., & Kittler, H. (2018).

The HAM10000 dataset, a large collection of multi-source dermatoscopic images of common pigmented skin lesions.
Scientific Data, 5, 180161.

[2] LeCun, Y., Bengio, Y., & Hinton, G. (2015).

Deep learning.
Nature, 521, 436–444.

[3] Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012).

ImageNet Classification with Deep Convolutional Neural Networks.
Advances in Neural Information Processing Systems.

[4] PyTorch Documentation.

<https://pytorch.org/docs/stable/index.html>

[5] HAM10000 Dataset on Kaggle.

<https://www.kaggle.com/datasets/kmader/skin-cancer-mnist-ham10000>

10. If working in a group, describe the work done by each member and state each person's contribution as a percentage of the total work.

Team Contribution

Member	Responsibility	Contribution
Thanutham Chonsongkram	Mainly responsible for dataset preparation, data preprocessing, CNN model implementation, training configuration, initial model evaluation. This included preparing the image dataset, implementing the custom PyTorch Dataset and DataLoader, designing the CNN architecture, setting up the loss function, optimizer, scheduler, and training loop.	50%
Tanapoom Srikaewkheaw	Mainly responsible for baseline model development, result comparison, evaluation interpretation, report writing, README.md preparation, GitHub documentation. This included implementing and analyzing the Random Forest baseline, comparing the CNN model with the baseline model, preparing evaluation tables and discussion, explaining the model architecture and code workflow, organizing the final documentation.	50%